

Guía técnica — App CAA-TEA

Documento dirigido a **desarrolladores del equipo** que vayan a tocar código, levantar el entorno o extender la app. Para la guía completa de producto e instalación, ver [README.md](#) . Para la instalación paso a paso en Windows con VS Code, ver [SETUP.md](#) .

Stack técnico

Capa	Tecnología	Versión
Frontend	React + Vite + PWA	18.3 / 5.4
Estado cliente	Zustand (con <code>persist</code> middleware)	4.5
Routing	React Router DOM	6.24
Gráficas	Recharts	2.12
TTS	Web Speech API (nativa del navegador)	—
Backend	Node.js + Express (ESM modules)	≥18 / 4.19
Base de datos	SQLite vía <code>sql.js</code> (WASM, sin compilación nativa)	1.11
Auth	JWT + <code>bcryptjs</code> para PIN del adulto	9.0 / 3.0
Pictogramas	API REST de ARASAAC	v1

Importante: usamos `sql.js` en lugar de `better-sqlite3` precisamente para que funcione en **Windows sin necesidad de Python ni Visual Studio Build Tools**. Cualquier intento de cambiar a `better-sqlite3` rompe la instalación en máquinas Windows estándar.

Estructura del repositorio

```
caa-tea/
├─ package.json           # Scripts del workspace (raíz)
├─ README.md              # Guía completa (producto + instalación + arranque)
├─ DESARROLLO.md          # Este archivo
├─ SETUP.md               # Instalación paso a paso en Windows / VS Code
├─ caa_tea.db              # Base SQLite (se crea sola)
├─ sql/
│   └─ 001_schema_sqlite.sql # Esquema único de BD
├─ server/                 # — BACKEND —
│   └─ index.js            # Bootstrap Express + initDb()
│       └─ package.json
│           └─ db/
│               └─ sqlite.js # Wrapper sql.js compatible con la API de better-sqlite3
│                   └─ middleware/
│                       └─ auth.js # requireAuth, requireAdult (JWT)
│                           └─ routes/
│                               └─ auth.js # /api/auth/{child-login, adult-login}
│                                   └─ users.js # /api/users/{children, all-children, all-adults, event}
│                                       └─ phrases.js # /api/phrases (POST guarda + suma estrellas, GET lista)
```

```

| | └─ schedule.js          # /api/schedule/{:id/today, :id, :id/advance}
| | └─ emotion.js           # /api/emotion (POST + suma estrellas)
| | └─ dashboard.js         # /api/dashboard/:childId - 6 métricas
| | └─ arasaac.js           # /api/arasaac/{search,categories} - proxy a ARASAAC
| | └─ categories.js        # /api/categories - vocabulario editable
| | └─ progress.js          # /api/progress/{rules, :childId}
| └─ utils/
|   └─ levels.js            # Tablas de niveles + reglas de estrellas
|   └─ progress.js          # awardStars(), getProgress(), cálculo de racha
└─ client/                  # — FRONTEND —
  └─ package.json
  └─ vite.config.js         # Proxy /api → :3001 + PWA
  └─ index.html
  └─ src/
    └─ main.jsx             # Router + RequireAuth
    └─ store.js             # Zustand: token, user, progress, pendingChildSession
    └─ index.css            # Tokens + animaciones CSS
    └─ hooks/useTTS.js      # speak(), stop()
    └─ utils/api.js         # fetch wrapper con JWT
    └─ components/
      └─ SelectChild.jsx    # Pantalla inicial
      └─ ChildApp.jsx       # Shell del niño (sidebar + topbar + FAB ayuda)
      └─ shared/
        └─ PinGate.jsx      # Numpad PIN
        └─ RewardPopup.jsx  # Popup con confeti
      └─ modules/
        └─ PhraseBuilder.jsx # Constructor de frases
        └─ Anticipation.jsx  # Agenda visual
        └─ EmotionRegulator.jsx # Regulador emocional
        └─ Achievements.jsx  # Niveles + racha + colección
      └─ adult/
        └─ AdultApp.jsx     # Dashboard con 6 gráficas
        └─ ScheduleEditor.jsx # Edición de la agenda
        └─ VocabularyEditor.jsx # Edición de vocabulario por categorías

```

Instalación y arranque

Requisitos

- **Node.js** ≥ 18 (probado con 20.x y 24.x)
- **npm** ≥ 9
- Sistema operativo: Windows / macOS / Linux. En Windows **no hace falta** Python ni Visual Studio.

Pasos

```

# 1. Entrar a caa-tea/
cd caa-tea

```

```

# 2. Opción rápida (recomendada)

```

```
npm run install:all
```

```
# o, manualmente:  
cd server && npm install && cd ..  
cd client && npm install && cd ..
```

Variables de entorno (opcional)

server/.env:

```
PORT=3001  
JWT_SECRET=cambia_esto_en_produccion  
CLIENT_URL=http://localhost:5173
```

Si no existe `.env`, los valores por defecto son los anteriores. El propio `index.js` los aplica vía `process.env.X || 'default'`.

Arrancar (dos terminales)

```
# Terminal 1 - backend  
cd server  
npm run dev           # nodemon (auto-reload)  
# o:  
npm start             # node plano (producción)  
  
# Terminal 2 - frontend  
cd client  
npm run dev           # vite en :5173
```

Abre **`http://localhost:5173`**.

Build de producción del cliente

```
cd client  
npm run build          # genera client/dist/  
npm run preview        # sirve el bundle generado
```

Reset completo

Para volver a estado de fábrica:

```
# Para todos los procesos node (Windows)  
taskkill /F /IM node.exe  
  
# Borra la BD (se recreará con datos demo al volver a arrancar)  
del caa_tea.db          # Windows  
# rm caa_tea.db         # macOS / Linux
```

```
# (opcional) limpia la caché de pictogramas en el navegador:
# DevTools → Application → Local Storage → borrar "caa-pictos-v2"
```

Esquema de base de datos

Definido en `sql/001_schema_sqlite.sql`. Todas las tablas usan `IF NOT EXISTS` así que las migraciones aditivas no rompen nada.

users	
└─ id (TEXT PK)	UUID. IDs deterministas para los usuarios demo.
└─ name (TEXT)	
└─ role (CHECK 'child' 'adult')	
└─ pin_hash (TEXT)	bcrypt, solo adultos
└─ avatar_color (TEXT)	
└─ created_at, updated_at	
adult_child_links	N:M entre adultos y niños
└─ adult_id (FK users)	
└─ child_id (FK users)	
generated_phrases	Cada frase guardada por el niño
└─ id (TEXT PK)	
└─ user_id (FK users)	
└─ pictogram_ids (TEXT)	JSON: [{id, label, category}, ...]
└─ phrase_length (INTEGER)	
└─ phrase_text (TEXT)	
└─ created_at	
schedules	Agenda diaria (3 ranuras)
└─ id, child_id (FK users)	
└─ date	
└─ slot_now, slot_next, slot_later	(cada uno JSON {pictoId, label, imageUrl, completed})
└─ UNIQUE (child_id, date)	
emotional_logs	Cada vez que el niño identifica una emoción
└─ id, user_id	
└─ emotion (TEXT)	'feliz' 'triste' ...
└─ intensity (1-3)	
└─ strategy_chosen (TEXT)	NULL = el niño pidió ayuda
└─ created_at	
usage_events	Telemetría general
└─ id, user_id	
└─ event_type	'phrase_built' 'emotion_logged' 'schedule_advanced'
	'picto_response_ms' 'picto_removed' 'phrase_cleared'
	'session_start' 'session_end' 'help_requested'
└─ details (TEXT)	JSON con metadatos del evento
└─ created_at	
INDEX idx_events_user_type	para acelerar queries del dashboard

child_progress	Gamificación persistente
└─ child_id (FK users) PK	
└─ total_stars (INTEGER)	
└─ level (1..10)	
└─ last_active_date	'YYYY-MM-DD'
└─ streak_days	
└─ updated_at	

IDs deterministas en seed

Para que `PinGate` no necesite endpoint de adultos antes del login, los usuarios demo se siembran con UUIDs fijos:

- **Adulto:** 000000000-0000-0000-0000-000000000001 (PIN: 1234)
- **Niño:** 000000000-0000-0000-0000-000000000010

Si rellenas la BD desde otro sitio, asegúrate de mantener este criterio o cambia `PinGate.jsx` para que tire del nuevo endpoint `/api/users/all-adults`.

API REST completa

Todas las rutas que requieren auth llevan `Authorization: Bearer <jwt>` en el header. El JWT lleva `{id, name, role}` y caduca a las 12h (niño) o 4h (adulto).

Auth

Método	Ruta	Body	Auth	Devuelve
POST	/api/auth/child-login	{childId}	no	{token, user}
POST	/api/auth/adult-login	{userId, pin}	no	{token, user}

Users

Método	Ruta	Auth	Notas
GET	/api/users/all-children	no	Lista pública para <code>SelectChild</code>
GET	/api/users/all-adults	no	Lista pública para <code>PinGate</code>
GET	/api/users/children	adulto	Hijos vinculados al adulto autenticado
POST	/api/users/event	sí	{event_type, details?} — telemetría libre

Phrases

Método	Ruta	Body	Devuelve
POST	/api/phrases	{pictoqramIds: [{id,label,category}], phraseText}	{id, progress: {total_stars, level, gained, ...}}
GET	/api/phrases?childId=&limit=	—	Array de frases recientes

Schedule

Método	Ruta	Auth	Notas
GET	/api/schedule/:childId/today	sí	Devuelve la agenda de hoy con slots parseados
PUT	/api/schedule/:childId	adulto	Sobrescribe los 3 slots
PATCH	/api/schedule/:childId/advance	sí	Marca slot_now como completed y devuelve los 3 slots + progress

Emotion

Método	Ruta	Body	Devuelve
POST	/api/emotion	{emotion, intensity, strategyChosen?}	{id, progress} (si hay estrategia)
GET	/api/emotion?childId=	—	Últimos 50 logs

Dashboard

```
GET /api/dashboard/:childId?weeks=8&days=30
Authorization: Bearer <adult_jwt>
```

Devuelve un objeto con las 6 series:

```
{
  "communicationProgress": [{ "week": "2026-16", "pictos": 12, "words": 4 }],
  "appUsage": {
    "daily": [{ "date": "2026-04-26", "events": 7, "duration_sec": 420 }],
    "weekly": [{ "week": "2026-16", "events": 42, "duration_sec": 2430 }],
    "monthly": [{ "month": "2026-04", "events": 120, "duration_sec": 7800 }]
  },
  "communicationType": [{ "week": "2026-16", "single": 3, "multi": 4 }],
  "errorsAndTime": {
    "responseMsByWeek": [{ "week": "2026-16", "avgMs": 850, "samples": 12 }],
    "errorCount": 5,
    "pictosRemoved": 4,
    "phrasesCleared": 1
  },
  "socialInteraction": [{ "week": "2026-16", "social": 3, "total": 12 }],
  "topPictograms": [{ "id": 4625, "label": "desayunar", "count": 4, "imageUrl": "...", "cate
  "emotionalAutonomy": { "total_logs": 7, "autonomous_logs": 5, "autonomy_rate": 71.4 }
}
```

Progress

Método	Ruta	Auth	Notas
GET	/api/progress/rules	no	Tabla de niveles y reglas de estrellas (público para mostrar en Achievements)
GET	/api/progress/:childId	sí	{total stars, level, streak_days, last_active_date, next_threshold}

ARASAAC (proxy)

Método	Ruta	Notas
GET	/api/arasaac/search?q=<keyword>&lang=es	Devuelve hasta 30 resultados normalizados {id, label, imageUrl, category}
GET	/api/arasaac/categories?lang=es	Lista de categorías ARASAAC

Categories (vocabulario editable)

Método	Ruta	Auth	Notas
GET	/api/categories	no	Categorías + keywords personalizables que alimentan PhraseBuilder
PUT	/api/categories/:id	adulto	Edita las keywords de una categoría

Salud

Método	Ruta	Notas
GET	/api/health	{ok:true, db:'SQLite'}

Sistema de niveles (cómo extenderlo)

Definido en `server/utils/levels.js`:

```
LEVEL_THRESHOLDS = [0, 25, 60, 110, 180, 270, 380, 520, 700, 1000];
STAR_RULES = [
  { action:'phrase_1',      stars: 2  },
  { action:'phrase_2_3',    stars: 5  },
  { action:'phrase_4_plus', stars: 8  },
  { action:'emotion_strategy', stars:10 },
  { action:'schedule_advance', stars:15 },
  { action:'streak_bonus',  stars:20  },
];
```

Para cambiar la curva basta con tocar `LEVEL_THRESHOLDS` . Para añadir una nueva acción que dé estrellas:

1. Define la regla en `STAR_RULES` (esto la mostrará en `Achievements`).
2. En la ruta backend correspondiente, llama a `awardStars(userId, n)` desde `server/utils/progress.js` .
3. Devuelve `{progress}` en la respuesta para que el cliente lo refleje.

`awardStars` se ocupa de:

- Crear la fila de `child_progress` si no existe.
- Recalcular la racha (consecutivo si era ayer, reset a 1 si no).
- Aplicar el **bonus de día completo** automáticamente cuando se llega a la 3ª acción del día.
- Recalcular el nivel con `levelFromStars()` .

Pictogramas: cómo se resuelven

Antes el código tenía ~230 IDs ARASAAC hardcodedos, muchos secuenciales o duplicados, sin garantía de que la imagen coincidiera con la etiqueta. Resultado: pictogramas mal colocados.

Ahora (en `client/src/components/modules/PhraseBuilder.jsx`):

- 1. Cada categoría declara solo *keywords* en español (sin IDs):

```
{ id:'comida', keywords:['desayunar','comer','beber','agua','leche', ...] }
```

- 2. Al pulsar una categoría por primera vez, el cliente lanza `Promise.all` (en lotes de 4) contra `/api/arasaac/search?q={keyword}` y se queda con el primer resultado.
- 3. Resultado se cachea en `localStorage` (clave `caa-pictos-v2`). Próximos arranques: instantáneo.
- 4. Para invalidar la caché tras cambios de keywords: incrementa el sufijo (`-v3`) y se rehará.

Esto garantiza que la imagen siempre se corresponda con la etiqueta porque el ID viene del propio motor de búsqueda de ARASAAC.

Telemetría disparada por el frontend

Evento	Cuándo se dispara	Sirve para
<code>session_start</code>	<code>ChildApp mounta</code>	Métrica de uso
<code>session_end</code>	<code>handleLogout</code> (con <code>duration_sec</code>)	Minutos de app
<code>phrase_built</code>	Cada <code>POST /phrases</code> (auto desde el backend)	Gráfica 1 y 3
<code>emotion_logged</code>	Cada <code>POST /emotion</code> (auto)	Autonomía
<code>schedule_advanced</code>	Cada <code>PATCH /advance</code> (auto)	Métrica rutina
<code>picto_response_ms</code>	Tiempo entre que se muestra el grid y se elige el primer picto	Gráfica 4
<code>picto_removed</code>	Niño borra un picto recién añadido	Errores
<code>phrase_cleared</code>	Niño pulsa la papelera con frase no vacía	Errores
<code>help_requested</code>	Niño pulsa el FAB de adulto	Métrica auxiliar

Todos van a la tabla `usage_events` con `event_type` y `details` JSON.

Smoke test manual (desde curl)

```
CHILD="00000000-0000-0000-0000-000000000010"
ADULT="00000000-0000-0000-0000-000000000001"

# 1. Health
curl http://localhost:3001/api/health

# 2. Login niño
TOKEN=$(curl -s -X POST http://localhost:3001/api/auth/child-login \
```



```
-H 'Content-Type: application/json' -d '{"childId\":"$CHILD\'}" \
| node -e "let d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>console.log(d))";

# 3. Progreso inicial
curl -H "Authorization: Bearer $TOKEN" http://localhost:3001/api/progress/$CHILD

# 4. Guardar frase de 4 pictos (debe dar +8)
curl -X POST -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"pictogramIds":[{"id":2510,"label":"Desayunar"}, {"id":4881,"label":"Leche"}, {"id":6459,"label":"Ver la tele"}, {"id":1234,"label":"Comer"}]}' \
http://localhost:3001/api/phrases

# 5. Login adulto
ATOK=$(curl -s -X POST http://localhost:3001/api/auth/adult-login \
-H 'Content-Type: application/json' -d '{"userId\":"$ADULT\","pin\":"1234"}' \
| node -e "let d='';process.stdin.on('data',c=>d+=c);process.stdin.on('end',()=>console.log(d))");

# 6. Dashboard completo
curl -H "Authorization: Bearer $ATOK" \
"http://localhost:3001/api/dashboard/$CHILD?weeks=8&days=30"
```

Troubleshooting

Error Cannot find package 'better-sqlite3'

Estás en una rama vieja. Hicimos migración a `sql.js`. Borra `node_modules` del server y reinstala:

```
cd server && rm -rf node_modules package-lock.json && npm install
```

node-gyp falla pidiendo Python

No deberías ver esto con la implementación actual. Si lo ves, es porque alguien volvió a `better-sqlite3`. Revisa `server/package.json` — debe tener `sql.js`, no `better-sqlite3`.

"PIN incorrecto" pero estoy poniendo 1234

El seed sembró un UUID aleatorio para el adulto en una versión antigua. Borra `caa_tea.db` y reinicia el server: el nuevo seed usa IDs deterministas.

Las imágenes de los pictogramas no aparecen

1. Verifica que el server esté arriba (`/api/health`).
2. Verifica que el proxy de Vite funcione: `curl http://localhost:5173/api/arasaac/search?q=comer` debería devolver JSON.
3. Limpia el cache: en DevTools → Local Storage → borra `caa-pictos-v2` y recarga.

El dashboard del adulto sale vacío

Es normal hasta que el niño haya creado al menos una frase. Loguéate como Mateo, crea 2-3 frases y vuelve.

Cambié las reglas de estrellas y siguen las antiguas

Las reglas se sirven desde `/api/progress/rules`. Reinicia el server. El cliente las recarga al montar `Achievements`.

El sistema de racha no se reinicia

El bonus de día completo y la racha se calculan dentro de `awardStars`. Si modificas la lógica, reinicia el server. Para resetear el progreso de un niño puntual:

```
UPDATE child_progress SET total_stars=0, level=1, streak_days=0, last_active_date=NULL WHERE
```

Tareas abiertas / siguientes pasos

- [] Pantalla de **alta de niños y adultos** desde el dashboard (hoy se hace insertando en BD).
- [] **Cambio de PIN** desde el panel del adulto.
- [] **Exportación CSV** de los datos del dashboard para informes terapéuticos.
- [] **Internacionalización** (hoy todo está en español hardcoded).
- [] **Modo offline completo**: usar Service Worker para cachear el bundle JS y operar sin red — el PWA ya tiene la base, hay que cubrir más rutas.
- [] **Tests automáticos** end-to-end con Playwright (cubrir el guion de demo del README).
- [] **Subir vista del adulto a tablet** (responsive ya, pero falta optimizar el dashboard a portrait).

Licencia y créditos

- **ARASAAC**: pictogramas Creative Commons By-NC-SA. Hay que mantener la atribución cuando se distribuya la app.
- **Web Speech API**: nativa del navegador, sin coste ni servidores.
- **Código del proyecto**: trabajo académico (curso 2025-2026, GIS).